Aggressive vs. Conservative TTG

Latest accepted results: Access coverage and unnecessary full-node fetches

JDrive

Aggressive: at least 832
unnecessary full-node fetches.
Conservative: 0 confirmed
in the audited classes.

LittleX

357 prefetched and used
in both policies.
Access-only reads are
covered by Conservative.

RecipePantry

Conservative: 118 / 164 covered
71.95% demand coverage
0 unused prefetches
0 confirmed white reads

GIL-off threads • n = 3 per reported configuration • native results preserved

RecipePantry Conservative uses the November 11 condition-call repair.
The other configurations retain their November 10 captures; Aggressive was not rerun.

Count white reads against actual Access demand

Using only an ID is valid demand. Visiting a node is not required for its Access to be useful.

Valid Access demand

The user query needs the node.
Reading jid(n) counts too.
A helper need not visit the node for its Access to count.

Filtered candidates

The node is outside the explicit graph-query filter:
e.g. a cooked Meal when status == planned is required.

Unused prefetch

Prefetched payload never read by foreground code.
This is one measurable class of unnecessary fetch.

Observed coverage = demanded IDs returned by prefetch / all demanded IDs

Counts are unique within each HTTP request / Session, then summed over the workflow.
The denominator includes root reads and memory hits. This is not a database cache-hit rate.

Confirmed white reads = unused + disjoint, excluded graph-query candidates.
Returning only IDs instead of full properties is a separate projection optimization.

Fixed workloads and explicit measurement provenance

The summary combines accepted captures; it is not a simultaneous new three-app rerun.

App	Business workflow	Capture used
JDrive	List / visible tree / trash / create folder	Nov 10: both policies
LittleX	Feed / likes / follow / profile / tweet	Nov 10: both policies
RecipePantry	Plan / buy / cook / boards / shopping	Nov 10 Agg.; Nov 11 Cons.

Same RecipePantry scenarios and business results

The repair changes Access extraction; native control flow, responses and persisted state agree.

Thread executor, GIL disabled, 1 worker, limit 2048. Each reported configuration has n=3.
The Nov 11 validation ran 8 fresh workflows: 2 excluded warmups + 6 measurements.
Only the repaired Conservative results are presented here.

Aggressive vs. Conservative: summary

Hit rate means demand coverage; white reads fall outside actual Access demand.

App	Aggressive hit rate	Conservative hit rate	Aggressive unnecessary fetches	Conservative unnecessary fetches
JDrive	91.40%	92.01%	At least 832	0 confirmed
LittleX	92.97%	92.97%	0 confirmed	0 confirmed
RecipePantry	67.07%	71.95%	At least 3	0 confirmed

RecipePantry Conservative audited: 118 prefetched / 118 used / 0 confirmed white reads.

RecipePantry Aggressive is the earlier 112 / 167 capture, not a rerun on the repaired version.
White reads count unnecessary nodes; valid jid-only accesses are not counted as waste.

Request-local unique IDs summed per workflow; all three repetitions agree.

Payload counts: use is not the same as necessity

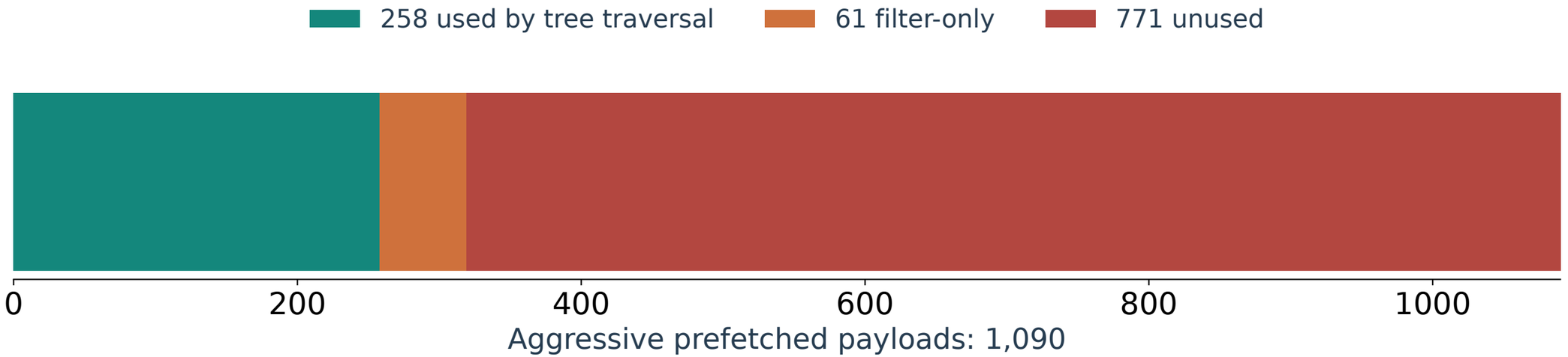
Latest accepted counts per configuration; all three repetitions agree

App / policy	Prefetched	Used	Unused	Filter-only	Unnecessary lower bound
JDrive / Agg.	1090	319	771	61	832 min.
JDrive / Cons.	265	265	0	0	0 found
LittleX / Agg.	357	357	0	0	0 found
LittleX / Cons.	357	357	0	0	0 found
RecipePantry / Agg.	112	112	0	3	3 min.
RecipePantry / Cons.	118	118	0	0	0 found

RecipePantry Conservative: all 118 prefetched objects were subsequently demanded.
The unfiltered _owns query intentionally reads neighbor IDs; these are valid Accesses.
No prefetched Meal was excluded by the planned-status graph-query filter in ShoppingList.

JDrive: unused and filter-only payloads

The trashed == False predicate avoids full-node fetches for rejected candidates.



Aggressive: at least 832

771 unused + 61 rejected trashed candidates with avoidable full payloads.

Conservative: 265 used

258 tree nodes + 7 ListFolder Accesses.
0 unnecessary examples in audited classes.

November 10 capture retained. Matched business outputs and final states; no new JDrive run.

LittleX: Access-only demand is real demand

357 prefetched and used in both policies; 92.97% observed coverage



Conservative payload use	Count
Prefetched and entered	287
Useful without Visit	70
Total used prefetch	357

A helper read is a valid demand even if the walker never visits the object.
November 10 accepted capture retained; no new LittleX run.

RecipePantry: repaired Conservative results

Condition calls and ordinary calls now share Access extraction; GIL-off threads, n=3.

Measure	Current result
Observed foreground demand	164
Prefetched / subsequently used	118 / 118
Observed demand coverage	118 / 164 = 71.95%
Unused prefetch / uncovered demand	0 / 46
Data / control / total SQL	213 / 166 / 379

All counts agree across the three measured workflows. Business outputs and persisted state pass.
All n3 audits find 0 unused and 0 excluded graph-query candidates. ID-only demand counts.
These precision captures alone do not establish a latency speedup.

One Access analysis for helpers and conditions

Prefetch supported reads when their real dependencies are available; keep control flow native.

1 Bind the object

meal = jobj(self.meal_id)
Native lookup materializes the actual Meal object.

2 Prefetch helper Access

Before _owns(meal, Meal):
check the actual safe type;
prefetch [root-->] demand.

3 Follow native control

Execute _owns once.
After ownership / status:
prefetch _requirements.

```
isinstance(item, kind) and jid(item) in [jid(n) for n in [root-->]]
```

Short-circuit guards remain part of the demand

A false AND-left or true OR-left prevents right-side prefetch; opaque predicates run natively.

Unknown predicate results do not discard a supported helper Access prefix.
Hooks can resume at supported statement boundaries after actual local bindings become available.

SQL counts are a separate outcome

Latest accepted data / control counts; request-local object coverage is not SQL hit rate.

App	Aggressive data / control	Conservative data / control
JDrive	208 / 52	53 / 54
LittleX	74 / 70	73 / 71
RecipePantry	189 / 150	213 / 166

RecipePantry covers 118 demands with 213 data SQL calls

Access extraction supplies demand information; it does not fuse every path or payload query.

RecipePantry Aggressive is an earlier capture; Conservative is the repaired November 11 version.
A Session demand can be satisfied from memory. A higher coverage ratio does not by itself
prove fewer SQL calls, less semantic waste, or lower latency.

What to improve next

Optimize the demand representation as well as the number of demands covered.

1 Optimize field projection separately

Ownership membership uses IDs. Returning fewer properties can save bytes without changing white-read counts.

2 Cover the remaining actual demand

RecipePantry still has 46 uncovered request-local demand IDs. Preserve control-flow guards.

3 Fuse compatible accesses and measure overhead

Track path / payload / control SQL separately; re-test request wall time with matched instrumentation.

4 Keep demand and waste measurements distinct

Report coverage, unused nodes and excluded query candidates; do not label valid ID-only demand as waste.

Evidence and scope

Current results are bound to accepted sources, captures and reproducible derivation.

RecipePantry repair: 8 fresh workflows, 6 measured

3 measured per version plus excluded warmups; only repaired results are shown in this deck.

Validation: 61 pure tests + 89 native regression tests

Native outputs / state / restart / GIL-off and all eight owned database cleanup checks passed.

Raw-derived results, with explicit provenance

256 new sealed files checked; all n3 counts match. Original three-app evidence is preserved.

Editable render_v4.py + accepted results + per-object necessity-audit.json evidence.
The new raw archives remain in the measurement workspace; the package retains its original 24 archives.
RecipePantry white reads audited across all n3. Aggressive / JDrive / LittleX were not rerun.